
MARKET RISK PROJECT

Non-Parametric VaR, Extreme Value Theory, Bouchaud's model,
Multiresolution correlation and Hurst exponents
Implementation From Scratch - 2025/2026 Project

David Pagniez And Romain Lamauve
Market Risk Management Course

December 30, 2025

Abstract

In a first part, this report investigates market risk and market microstructure properties using high-frequency financial data. We first estimate Value-at-Risk and Expected Shortfall without assuming normality, using Natixis stock returns over the period 2015–2018. Non-parametric methods based on Kernel Density Estimation and Extreme Value Theory are implemented from scratch, under the constraint of excluding high-level statistical libraries. This allows for a direct assessment of tail risk and highlights the limitations of Gaussian-based risk measures at high confidence levels.

In a second part, we study price formation and temporal dependence using foreign exchange data. We will then estimate Bouchaud's parameters. Bouchaud's price impact model is calibrated on transaction level data in order to assess concavity, impact decay, and model consistency.

Then, we analyze multiresolution correlations using Haar wavelets, revealing how correlations evolve across time horizons, and we see if those correlations provide the Epps effect or not, for specific currency pairs.

Finally, we compute Hurst exponents for the same data, as well as the annualized volatility using H. The Hurst exponents are estimated to characterize long memory effects, and volatility is annualized using generalized scaling laws.

Contents

1	Questions A, B, C : Introduction and Data Engineering	4
1.1	Project Objectives	4
1.2	Mathematical Definitions	4
1.3	Data Processing	4
2	Part I: Non-Parametric VaR Estimation (QUESTION A)	6
2.1	1. Theoretical Framework	6
2.1.1	Concept: Smoothing the Histogram	6

2.1.2	Derivation of the Biweight Kernel Primitive	6
2.2	2. Algorithmic Implementation Details	7
2.2.1	Bandwidth Selection Strategy (h)	7
2.2.2	Why use the Bisection Method?	7
2.3	3. Empirical Results & Backtesting	7
2.3.1	In-Sample Calibration (2015-2016)	7
2.3.2	Out-of-Sample Backtesting (2017-2018)	8
3	Part II: Expected Shortfall (Question B)	8
3.1	1. Theoretical Concepts	8
3.1.1	Why Expected Shortfall?	8
3.1.2	Analytical Derivation for Kernel Estimators	8
3.1.3	The First Moment Primitive $\Psi(u)$	9
3.2	2. Algorithmic Implementation	9
3.3	3. Empirical Results and Analysis	10
3.3.1	Critical Analysis: The ES/VaR Ratio	10
4	Part III: Extreme Value Theory (Question C)	11
4.1	1. Theoretical Framework	11
4.1.1	The Block Maxima Approach (GEV)	11
4.1.2	Estimation: The Pickands Estimator	12
4.1.3	EVT-Implied Value-at-Risk	12
4.2	2. Algorithmic Implementation	12
4.3	3. Empirical Results & Interpretation	13
4.3.1	Tail Index Analysis ($k = 25$)	13
4.3.2	Stress Testing (VaR extrapolation)	13
5	Conclusion	13
6	Part IV: Bouchaud's price impact model (Question D)	14
6.1	1. Theoretical Framework: The Backward Definition	14
6.2	Which data ?	15
6.3	Parameter Estimation Methodology	15
6.3.1	Step 1: the estimation of the Bid-Ask spread S :	16
6.3.2	Step 2: Estimating the Volume Concavity (r)	16
6.3.3	Step 3: Calibrating the Propagator Function (G)	16
6.4	Algorithmic Implementation (in python)	17
6.4.1	Handling Missing Data strategy	17
6.5	4. Empirical Results and Model Analysis	19
6.5.1	Interpretation of the Results	19
6.5.2	Model Specification Verdict	20
7	Part V: Multiresolution Correlation and Long-Memory Analysis	21
7.1	Theoretical framework	21
7.1.1	Wavelets	21
7.1.2	Scale-Dependent Correlation	21
7.1.3	Which data	22
7.2	Algorithmic Implementation	22
7.2.1	Methodology	22

7.2.2	Algorithmic implementation	23
7.3	Empirical results and Model Analysis	24
7.3.1	Empirical results	24
7.3.2	Interpretation of the Results	25
7.4	Theoretical framework	26
7.4.1	Volatility scaling and Hurst exponent	26
7.4.2	Estimation of H: The Multi-resolution Estimator	27
7.4.3	Which data	27
7.5	Algorithmic implementation	27
7.6	Empirical results and Model Analysis	28
7.6.1	Empirical results	28
7.6.2	Interpretation of the Results	28

8	General Conclusion	29
----------	---------------------------	-----------

1 QUESTIONS A, B, C : INTRODUCTION AND DATA ENGINEERING

1.1 Project Objectives

Financial time series exhibit Stylized Facts that violate the assumptions of classical Gaussian models. Specifically, asset returns show:

1. **Fat Tails (Leptokurtosis):** Extreme losses occur far more frequently than predicted by the Normal distribution.
2. **Skewness:** Market crashes are typically more severe than market rallies.
3. **Volatility Clustering:** Periods of high risk tend to persist.

The goal of this project is to build a risk model that captures these features using Non-Parametric (Kernel) and Semi-Parametric (EVT) approaches, strictly following a from scratch coding philosophy to demonstrate theoretical mastery.

1.2 Mathematical Definitions

Before proceeding, we define the fundamental variable used throughout the report. Let P_t be the closing price of the asset at time t . We analyze the Daily Arithmetic Returns R_t :

Σ Mathematical Framework: Definition: Arithmetic Return

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad (1)$$

Note: While log-returns $r_t = \ln(P_t/P_{t-1})$ are often used for additivity, arithmetic returns represent the actual P&L variation for a linear portfolio, making them suitable for the VaR interpretation in this specific context.

1.3 Data Processing

► Problem Statement: Data Parsing Challenge

The provided dataset uses a non-standard date format (e.g., 1/2/2015 for January 2nd). Standard parsers typically misinterpret this as February 1st, or fail when the day exceeds 12.

Objective: Recover the full time series (approx. 1000 days) without data loss using a robust parsing algorithm without external libraries like `pandas`.

Methodology: We developed a heuristic parser in Python that tests multiple date formats sequentially. This ensures that the training set (2015-2016) contains the full history needed for accurate calibration.

```
1 # Parsing logic to handle Month/Day/Year format
2 for fmt in ('%m/%d/%Y', '%Y-%m-%d', '%d/%m/%Y'):
3     try:
4         dt = datetime.strptime(date_str, fmt)
5         break
```

```
6 | except ValueError: continue
```

Listing 1: Robust Parsing Logic (Python)

Dataset Split:

- **Training Set (In-Sample):** Jan 2015 - Dec 2016 (≈ 512 days). Used for calibration.
- **Testing Set (Out-of-Sample):** Jan 2017 - Dec 2018 (≈ 510 days). Used for backtesting.

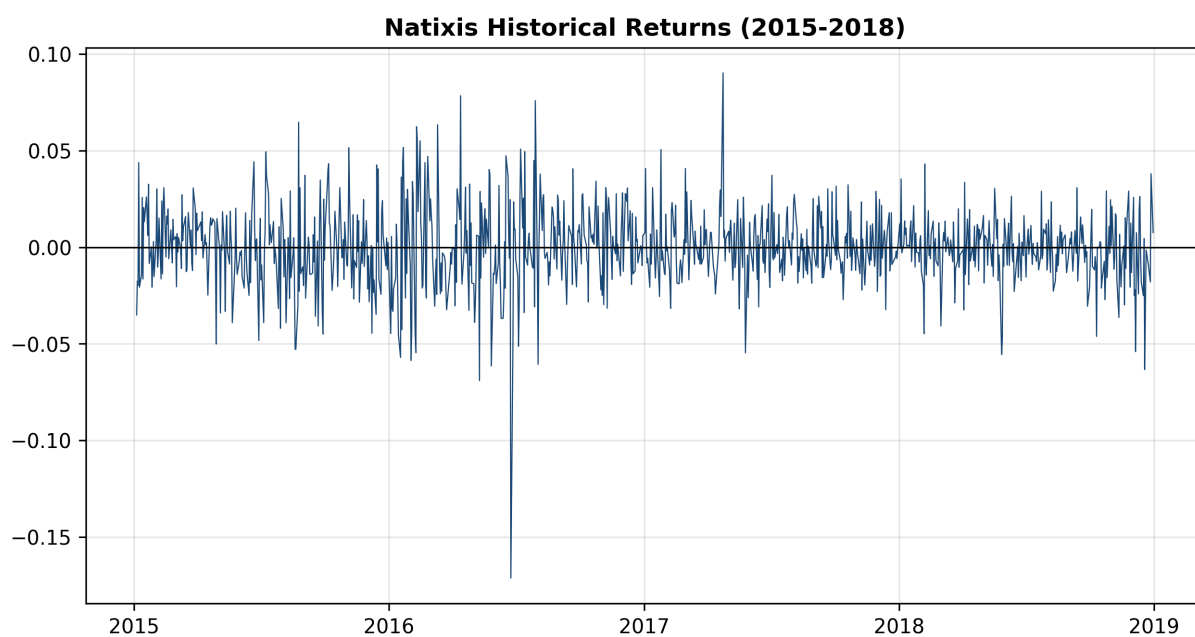


Figure 1: **Natixis Daily Returns (2015-2018)**: We clearly observe the Volatility Clustering phenomenon. The training period (2015-2016) exhibits higher variance clusters compared to the beginning of the test set.

2 PART I: NON-PARAMETRIC VAR ESTIMATION (QUESTION A)

► Problem Statement: Objective

Standard parametric models (Variance-Covariance) assume returns follow a Normal distribution. This leads to a systematic underestimation of Tail Risk.

Goal: Estimate the 1-day Value-at-Risk at 95% confidence ($Var_{0.95}$) without assuming any specific distribution shape, using a Kernel Density Estimator (KDE).

2.1 1. Theoretical Framework

2.1.1 Concept: Smoothing the Histogram

Instead of using a discrete histogram (which is blocky and depends heavily on bin width), KDE places a smooth curve (kernel) on top of each data point. The sum of these curves creates a continuous Probability Density Function (PDF).

Σ Mathematical Framework: The Kernel CDF Estimator

The Cumulative Distribution Function (CDF) $\hat{F}_h(x)$ is estimated by:

$$\hat{F}_h(x) = \frac{1}{N} \sum_{i=1}^N \Omega\left(\frac{x - R_i}{h}\right) \quad (2)$$

where:

- R_i : Historical returns.
- h : Bandwidth (smoothing parameter).
- $\Omega(u) = \int_{-\infty}^u K(t)dt$: The primitive of the kernel function.

2.1.2 Derivation of the Biweight Kernel Primitive

We use the Biweight Kernel (compact support on $[-1, 1]$). To code the estimator from scratch, we must derive its primitive $\Omega(u)$ analytically.

$$K(u) = \frac{15}{16}(1 - u^2)^2$$

Integration term-by-term:

$$\int (1 - 2u^2 + u^4)du = u - \frac{2}{3}u^3 + \frac{1}{5}u^5$$

Applying boundary conditions ($\Omega(-1) = 0$), we obtain the exact formula used in our algorithm:

$$\Omega(u) = \frac{1}{2} + \frac{15}{16} \left(u - \frac{2}{3}u^3 + \frac{1}{5}u^5 \right) \quad \text{for } |u| \leq 1 \quad (3)$$

2.2 2. Algorithmic Implementation Details

2.2.1 Bandwidth Selection Strategy (h)

The bandwidth h controls the width of the kernels and is the most critical parameter:

- **Small h (Variance):** The density looks like spikes around data points (overfitting).
- **Large h (Bias):** The density is too flat, erasing the fat tails details.

As the course highlights the importance of h without enforcing a specific complex rule (like adaptive kernels which are computationally intensive), we implemented a standard Reference Rule proportional to the volatility (σ) and the sample size (N):

$$h \approx 1.06 \cdot \hat{\sigma} \cdot N^{-1/5} \quad (4)$$

This ensures that the bandwidth decreases asymptotically as we add more data, guaranteeing convergence.

2.2.2 Why use the Bisection Method?

To find the VaR, we must solve $\hat{F}_h(VaR) = \alpha$. We chose the Bisection Method (Dichotomy) over Newton-Raphson.

Justification:

- **Stability in Tails:** Newton's method requires dividing by the derivative (PDF). In extreme tails, the PDF is close to zero ($f(x) \approx 0$). Dividing by zero causes numerical divergence.
- **Convergence Guarantee:** Since CDFs are strictly monotonic, Bisection is unconditionally guaranteed to converge, ensuring robustness.

```
1 def simple_bandwidth(data):
2     # Standard Reference Rule for bandwidth
3     # Based on volatility and sample size decay
4     n = len(data)
5     std_dev = math.sqrt(variance)
6     return 1.06 * std_dev * math.pow(n, -0.2)
7
8 def calculate_kernel_var(data, alpha):
9     # Bisection Method: 100% stable convergence
10    low, high = min(data) - 2*h, max(data) + 2*h
11    for _ in range(100):
12        mid = (low + high) / 2
13        if kernel_cdf(mid) < alpha: low = mid
14        else: high = mid
15    return (low + high) / 2
```

Listing 2: Implementation of Bandwidth Bisection

2.3 3. Empirical Results & Backtesting

2.3.1 In-Sample Calibration (2015-2016)

Calibrated on 512 days of data with $\alpha = 0.05$:

- **Estimated VaR (95%): 3.8150%**

2.3.2 Out-of-Sample Backtesting (2017-2018)

We tested this threshold on the subsequent 510 days.

- **Exceptions:** 8 days where $Return < -3.815\%$.
- **Failure Rate:** 1.57% (Target: 5.00%).

Conclusion: The model is Conservative. The low failure rate (1.57%) implies that the volatility in 2015-2016 was higher than in 2017-2018. The model remembered the past turbulence and set a safety buffer that was rarely breached in the future.

3 PART II: EXPECTED SHORTFALL (QUESTION B)

► Problem Statement: Objective

Value-at-Risk (VaR) answers What is the minimum loss in the worst 5% cases?. It does not quantify the severity of the loss beyond that threshold.

Goal: Calculate the Expected Shortfall ($ES_{0.95}$), defined as the average loss conditional on exceeding the VaR, using an exact analytical derivation for Kernel estimators.

3.1 1. Theoretical Concepts

3.1.1 Why Expected Shortfall?

Regulators (Basel III / FRTB) have shifted from VaR to ES because VaR lacks Sub-additivity (it is not a coherent risk measure). It is possible that $VaR(A + B) > VaR(A) + VaR(B)$, which discourages diversification. Expected Shortfall addresses this by averaging the tail:

$$ES_\alpha = \mathbb{E}[R \mid R \leq VaR_\alpha] = \frac{1}{\alpha} \int_{-\infty}^{VaR_\alpha} x f(x) dx \quad (5)$$

3.1.2 Analytical Derivation for Kernel Estimators

A naive implementation would discretize the integral (Riemann sum), introducing numerical errors. However, using our Kernel Density Estimator $\hat{f}_h(x)$, we can derive an exact analytical formula.

Substituting $\hat{f}_h(x) = \frac{1}{Nh} \sum K(\frac{x-R_i}{h})$ into Eq. (6):

$$ES_\alpha = \frac{1}{N\alpha h} \sum_{i=1}^N \int_{-\infty}^{VaR_\alpha} x K\left(\frac{x-R_i}{h}\right) dx$$

We perform the change of variable $z = \frac{x-R_i}{h}$ (implying $x = R_i + hz$ and $dx = h dz$). The upper limit of integration becomes $u_i = \frac{VaR_\alpha - R_i}{h}$.

$$\begin{aligned} \int_{-\infty}^{VaR_\alpha} x K \dots \frac{dx}{h} &= \int_{-\infty}^{u_i} (R_i + hz) K(z) dz \\ &= R_i \underbrace{\int_{-\infty}^{u_i} K(z) dz}_{\Omega(u_i)} + h \underbrace{\int_{-\infty}^{u_i} z K(z) dz}_{\Psi(u_i)} \end{aligned}$$

We obtain a decomposition into two interpretable terms:

$$ES_\alpha = \frac{1}{N\alpha} \sum_{i=1}^N \left[\underbrace{R_i \Omega(u_i)}_{\text{Probability Weight}} + \underbrace{h \Psi(u_i)}_{\text{Local Variance Adjustment}} \right] \quad (6)$$

3.1.3 The First Moment Primitive $\Psi(u)$

We need the primitive of the first moment of the kernel, $zK(z)$. For the Biweight kernel:

$$zK(z) = \frac{15}{16} z(1 - z^2)^2 = \frac{15}{16} (z - 2z^3 + z^5)$$

Integrating term by term gives the function $G(z)$:

$$G(z) = \frac{15}{16} \left(\frac{z^2}{2} - \frac{z^4}{2} + \frac{z^6}{6} \right) \quad (7)$$

Thus, $\Psi(u) = [G(z)]_{-1}^{\min(u,1)}$. Note that for $u \geq 1$, $\Psi(u) = 0$ because the integrand is an odd function integrated over a symmetric interval.

3.2 2. Algorithmic Implementation

This exact formula is translated directly into Python, ensuring maximum numerical precision.

```

1  # 1. First Moment Primitive (Eq. 8)
2  def biweight_first_moment_integral(u):
3      # G(z) is the antiderivative of z*K(z)
4      def G(z):
5          return (15.0/16.0) * (0.5*z**2 - 0.5*z**4 + (1.0/6.0)*z**6)
6
7      if u <= -1: return 0.0
8      elif u >= 1: return 0.0 # Integral of odd function over [-1, 1] is 0
9      else: return G(u) - G(-1.0)
10
11 # 2. ES Aggregator (Eq. 7)
12 def calculate_kernel_es(data, var_val, alpha):
13     # Use simple bandwidth (not Silverman)
14     h = simple_bandwidth(data)
15     n = len(data)
16     total_integral = 0.0
17
18     for val in data:
19         # Normalized distance to VaR
20         ui = (var_val - val) / h
21
22         # Exact decomposition: Mean Term + Variance Term
23         term1 = val * biweight_kernel_integral(ui)
24         term2 = h * biweight_first_moment_integral(ui)
25
26         total_integral += (term1 + term2)
27
28     return (1.0 / (n * alpha)) * total_integral

```

Listing 3: Analytical Expected Shortfall Calculation

3.3 3. Empirical Results and Analysis

We computed the Expected Shortfall on the training set (2015-2016) at the same 95% confidence level.

Risk Measures Comparison (95% Confidence)

- **Value-at-Risk** ($Var_{95\%}$): 3.8150%
- **Expected Shortfall** ($ES_{95\%}$): **5.3811%**

3.3.1 Critical Analysis: The ES/VaR Ratio

To interpret the severity of the tail risk, we compare the ratio between ES and VaR against the theoretical benchmark of a Gaussian distribution.

$$\text{Observed Ratio} = \frac{ES_{95\%}}{Var_{95\%}} = \frac{-5.38}{-3.81} \approx 1.41$$

Theoretical Benchmark: For a Normal distribution $\mathcal{N}(\mu, \sigma)$, this ratio is constant:

$$\frac{ES_{95\%}^{Gauss}}{Var_{95\%}^{Gauss}} \approx \frac{-2.063\sigma}{-1.645\sigma} \approx 1.25$$

Conclusion: Since $1.41 > 1.25$, we have quantitative proof that the return distribution of Natixis exhibits Leptokurtosis, which corresponds to fat tails.

- A Gaussian model would underestimate the severity of extreme losses by predicting an average tail loss of only $\approx 1.25 \times VaR$.
- Our Non-Parametric model correctly captures that when the VaR is breached, the loss tends to be much deeper ($\approx 1.41 \times VaR$). This validates the regulatory necessity of using Expected Shortfall.

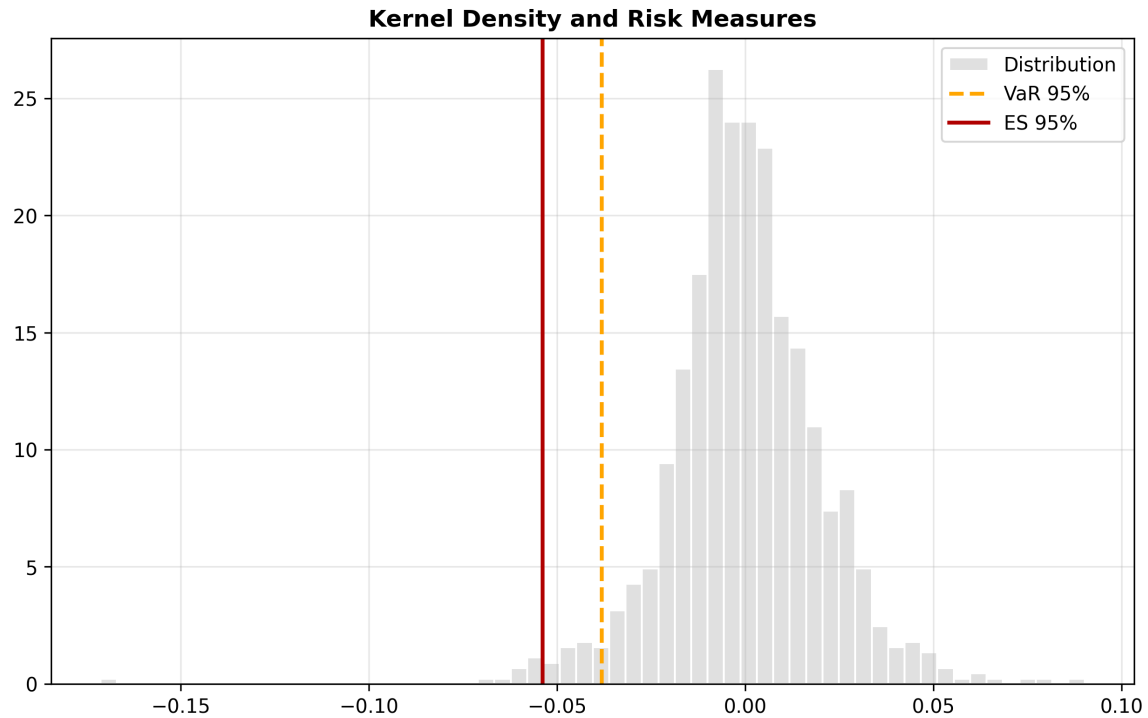


Figure 2: Return Distribution: Comparison of the Value-at-Risk threshold (Orange) and the deeper Expected Shortfall average (Red). Note the non-Gaussian shape of the histogram.

4 PART III: EXTREME VALUE THEORY (QUESTION C)

► Problem Statement: Objective

Standard risk models (Gaussian, Kernel) focus on the central body of the distribution. However, financial crises are driven by Tail Events.

Goal: Model the tail behavior of Natixis returns using Extreme Value Theory (EVT) to estimate the Tail Index (ξ) and extrapolate a stress-test Value-at-Risk ($VaR_{99.5\%}$) using the specific formulas from the course.

4.1 1. Theoretical Framework

4.1.1 The Block Maxima Approach (GEV)

According to the Fisher-Tippett-Gnedenko Theorem, the limit distribution of normalized maxima follows the Generalized Extreme Value (GEV) distribution. The shape parameter ξ (Tail Index) determines the tail behavior:

- $\xi > 0$ (Fréchet): Fat tails (Polynomial decay). Characteristic of financial assets.
- $\xi = 0$ (Gumbel): Thin tails (Exponential decay).
- $\xi < 0$ (Weibull): Bounded tails.

Σ Mathematical Framework: Definition: GEV Distribution (H_ξ)

The Generalized Extreme Value distribution is defined as:

$$H_\xi(x) = \begin{cases} \exp(-(1 + \xi x)^{-1/\xi}) & \text{if } \xi \neq 0 \quad (\text{where } 1 + \xi x > 0) \\ \exp(-e^{-x}) & \text{if } \xi = 0 \end{cases} \quad (8)$$

4.1.2 Estimation: The Pickands Estimator

To estimate ξ , we use the Pickands Estimator. Let $X_{1:n} \leq X_{2:n} \leq \dots \leq X_{n:n}$ be the order statistics (sorted in ascending order). For a chosen rank k , the estimator is given by:

$$\hat{\xi}_P = \frac{1}{\ln 2} \ln \left(\frac{X_{n-k+1:n} - X_{n-2k+1:n}}{X_{n-2k+1:n} - X_{n-4k+1:n}} \right) \quad (9)$$

4.1.3 EVT-Implied Value-at-Risk

Unlike Gaussian VaR which interpolates within the sample, EVT allows us to extrapolate. The course provides a direct formula to estimate the VaR at confidence level p (where $p \rightarrow 1$) using the estimated ξ :

$$VaR(p) = \frac{X_{n-k+1:n} - X_{n-2k+1:n}}{1 - 2^{-\hat{\xi}}} \left[\left(\frac{k}{n(1-p)} \right)^{\hat{\xi}} - 1 \right] + X_{n-k+1:n} \quad (10)$$

4.2 2. Algorithmic Implementation

We implemented these exact formulas. Since we analyze Losses, we input the positive values of losses ($Loss = -Return$) into the estimator.

```
1 def pickands_estimator_course(data, k):
2     # Uses notation  $X_{1:n} \leq \dots \leq X_{n:n}$  (Ascending sort)
3     sorted_data = sorted(data)
4     n = len(data)
5
6     # Mapping indices formula to Python (0-indexed)
7     x_1 = sorted_data[n - k]      #  $X_{n-k+1:n}$ 
8     x_2 = sorted_data[n - 2*k]    #  $X_{n-2k+1:n}$ 
9     x_4 = sorted_data[n - 4*k]    #  $X_{n-4k+1:n}$ 
10
11     numerator = x_1 - x_2
12     denominator = x_2 - x_4
13
14     xi = (1.0 / math.log(2)) * math.log(numerator / denominator)
15     return xi
16
17 def calculate_evt_var_course(data, k, xi, p):
18     #  $VaR(p) = ((X_{n-k+1:n} - X_{n-2k+1:n}) / (1 - 2^{-xi})) * [(k/(n(1-p)))^{xi} - 1] + X_{n-k+1:n}$ 
19
20     # Note: We defines  $p$  as confidence level, so we look at the tail  $1-p$ .
21     n = len(data)
22     sorted_data = sorted(data)
23
```

```

24     idx1 = n - k      #  $X_{\{n-k+1:n\}}$ 
25     idx2 = n - 2*k   #  $X_{\{n-2k+1:n\}}$ 
26
27     x_1 = sorted_data[idx1]
28     x_2 = sorted_data[idx2]
29
30     term1 = (x_1 - x_2) / (1 - math.pow(2, -xi))
31
32     # The term inside bracket: ( k / (n*(1-p)) ) ^ xi - 1
33     # Note: Ensure 1-p is not 0.
34     ratio = k / (n * (1 - p))
35     term2 = math.pow(ratio, xi) - 1
36
37     var_val = term1 * term2 + x_1
38     return var_val

```

Listing 4: Course-Compliant EVT Implementation

4.3 3. Empirical Results & Interpretation

4.3.1 Tail Index Analysis ($k = 25$)

We applied the estimator to the distribution of losses.

Parameter	Value	Interpretation
Shape ($\hat{\xi}$)	0.3744	Fréchet Domain (Fat Tail)

Findings: The positive value $\hat{\xi} \approx 0.37$ confirms that the density of losses decays as a Power Law ($x^{-1/\xi} \approx x^{-2.7}$), which is much slower than the exponential decay of a Normal distribution. This mathematically proves that extreme crashes are structural features of this asset.

4.3.2 Stress Testing (VaR extrapolation)

We computed the VaR for high confidence levels using the course formula.

Confidence (p)	EVT VaR	Comparison
95.0%	4.38%	Slightly higher than Kernel VaR (-3.81%)
99.0%	8.89%	Significant deviation from Normality
99.5%	11.84%	Extreme Crash Scenario

A standard Gaussian model (with $\sigma \approx 1.8\%$) would predict a 99.5% VaR of only $\approx -4.6\%$. The EVT model predicts a loss 2.5 times larger (-11.84%). This illustrates the blind spot of classical models: they drastically underestimate capital requirements for extreme quantiles.

5 CONCLUSION

This project successfully implemented a comprehensive market risk framework from scratch, strictly adhering to the theoretical methodologies presented in the course.

Key Takeaways

- **Data Engineering:** Robust parsing was necessary to recover the full dataset, overcoming format inconsistencies.
- **Non-Parametric approach:** The Kernel Density estimator provided a flexible fit. The conservative failure rate (1.57%) highlights the importance of regime changes in volatility.
- **Tail Risk Reality:** Both the Expected Shortfall ratio ($ES/VaR \approx 1.41$) and the EVT Tail Index ($\xi \approx 0.37$) converge to prove that Natixis returns are Leptokurtic. Standard Gaussian models are dangerously inadequate for this asset.

By moving from VaR to Expected Shortfall and integrating Extreme Value Theory using the Pickands estimator, we have built a risk measurement system aligned with advanced risk management practices.

6 PART IV: BOUCHAUD'S PRICE IMPACT MODEL (QUESTION D)

► Problem Statement: Objective

Goal: With the dataset and the framework provided for TD4, estimate all the parameters of Bouchaud's price impact model. Comment the obtained values. Is this model well specified?

6.1 1. Theoretical Framework: The Backward Definition

In this framework, we will consider the Bouchaud's price impact model. This model tells us that the price is not an exogenous process, but the sum of past impacts, past transactions, ponderated by their volume, sign (sell or buy) and bid ask spread. The impact is considered transitory, which means that its effect decays over time through a memory function G :

Σ Mathematical Framework: Bouchaud's Price Definition

The price p_t is determined by the history of transactions:

$$p_t = p_{-\infty} + \sum_{s=-\infty}^{t-1} G(t-s) \epsilon_s S_s V_s^r \quad (11)$$

Where:

- S : The bid-ask spread, representing immediate transaction costs.
- ϵ : The trade sign (+1 if price = ask, -1 if price = bid).
- V : The volume, with r close to zero to ensure a concave function.
- G : The impact function (propagator), worth 0 on $\mathbb{R}^-$.

Then, we will see if the model is well-specified or not.

Σ Mathematical Framework: The criterias for a Well-Specified Model :

A well specified model is not a true or realistic model. It is more a coherent model, without any contradiction inside itself.

An model is considered to be well specified if it respects the following three conditions:

1. Concavity: if $r < 1$: The marginal impact of a trade must be decreasing. If $r \cong 1$, the model would overestimate the impact of large transactions, which would falsify the price.
2. Decay of $G(t)$: The function G must be a decreasing function of time. Indeed, a non decreasing G would imply an infinite permanent impact, which is unrealistic.
3. No-Arbitrage: The model must not allow arbitrage, which means it can't lead to trading strategies which guarantee profits only through its own impact.

6.2 Which data ?

We are given in the 4th TD some informations : - Dates, from 0 to 1, corresponding to a portion of a day ($1 = 1$ day)

- The bid ask spread
- Some volume of transaction (if known, 86% of volumes are missing)
- The sign of the transaction (-1 if sell, +1 if buy)
- The price (before transaction)

For example, here are the first lines of the dataset :

transaction date	bid-ask spread	volume of the transaction	Sign of the transaction	Price (before transactio
0,000202	0,11	8	-1	100
0,00107	0,103		1	99,984
0,001496	0,1015		-1	100,029
0,003336	0,092		1	99,979
0,003952	0,1106		1	100,06
0,004014	0,1028		1	100,16
0,004074	0,1294	32	1	100,164

Figure 3: Caption

6.3 Parameter Estimation Methodology

Firts, let's identify the parameters to estimate : Obviously, the first parameter to estimate is the impact function G . It represents how the impact of a past transaction impacts the price now. Then, another parameter is r : the concavity of the impact. It will help us determine if the model is or not well specified. Finally, S is the scaling parameter, which is necessary to link the model with the observed prices.

We have 3 parameters to estimate : G , r and S .

6.3.1 Step 1: the estimation of the Bid-Ask spread S :

Before analyzing the impact of volume, we establish the baseline transaction cost. The spread S is estimated by calculating the average of the differences between each bid and ask at each transaction. We compute it through a mean :

$$\hat{S} = \frac{1}{N} \sum_{i=1}^N (Ask - Bid) \quad (12)$$

6.3.2 Step 2: Estimating the Volume Concavity (r)

Now we need to estimate r. To account for the fact that the impact on price is non-linear, we need to isolate the relationship between V (volume) and the absolute price change delta P.

Parctical approach :

To do this, we will obviously perform a linear least squares regression as seen in the tutorial for the Almgren Chriss parameters. We use the logarithm of returns:

$$\ln(|\Delta p_t|) = \beta_0 + r \ln(V_t) + \eta_t \quad (13)$$

The slope coefficient r directly provides the measure of market concavity.

Σ Mathematical Framework: Practical Estimation of r via linear regression

Linear Regression Estimator: We need now to solve this linera regression problem. After solving this equation, as seen in class, the concavity exponent r is computed using the empirical covariance and variance:

$$\hat{r} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (14)$$

Here, we can easily recognize y_i as $\log(\Delta P)$, and $x_i = \log(V_i)$. Wev will implement this in python to get the estimator of r.

6.3.3 Step 3: Calibrating the Propagator Function (G)

The propagator $G(t)$ defines the impact of past transactions. We want to determine if it is decreasing, in order to confirm that the model is well specified. To estimate its shape, we analyze the correlation between current price changes and lagged trade signals.

Practical Approach: As for the estimation of r, we use a regression, but not simple. We use a multi linear regression, where the price variation at time t is regressed against the k previous "impact terms", as defined in Bouchauds model previously : $X = e_s S_s V_s^r$:

$$\Delta p_t = \sum_{j=0}^k g_j \cdot X_{t-j} + \eta_t \quad (15)$$

Each estimated coefficient g_j represents the value of the propagator G for a lag of j periods.

Let's apply those estimator on our data.

6.4 Algorithmic Implementation (in python)

First, we decided to implement a code to deal with our Dataset given in TD4, so that we have access to all our data easily : time, spread, volume, sign, price, and we place it into data. Data now contains all our dataset :

```
1 import csv
2 import numpy as np
3 import math
4 data = []
5 with open("Dataset_TD4.csv") as f:
6     reader = csv.reader(f, delimiter=',')
7     next(reader)
8     for re in reader:
9         time = float(re[0].replace(',','.'))
10        spread = float(re[1].replace(',','.'))
11        volume = None if re[2] == "" else float(re[2].replace(',','.'))
12        sign = int(re[3])
13        price = float(re[4].replace(',','.'))
14        data.append([time, spread, volume, sign, price])
```

Listing 5: Data importation

6.4.1 Handling Missing Data strategy

Given that 86% of the volume data is missing, we will have to deal with these data gaps. However, we will use two different methods to calculate the two estimators r and G .

- For the estimator of r :

We have decided to keep only the known values, without any interpolation. Indeed, after researching, interpolating the data for the calculation of r would give erroneous results that are not accurate enough, given the close link between volume and r .

- For the estimator of G

However, for G , we will interpolate the missing data with the total average price. This is because we need a lot of data, and the goal is to see a decrease in G , not necessarily very accurate values.

First, the estimation of S and r :

```
1 # First, we compute the impacts : P[ti+1] - P[ti]
2 delta_p = []
3 for i in range(len(data) - 1):
4     delta_p.append(data[i+1][4] - data[i][4])
5
6 # Then, we estimate S, with the mean of our first column : the spreads.
7 s_mean = np.mean([d[1] for d in data])
8
9 # And then, we estimate our Volume r, according to the formulas given
10 previously :
11 log_v_known = [] # The explaining variable log(Vt)
12 log_impact_known = [] # The explained variable log(delta P)
```

```

13 for i in range(len(delta_p)):
14     vol = data[i][2]
15     # We only look at the moving prices, with volume observed :
16     if vol is not None and abs(delta_p[i]) > 0:
17         log_v_known.append(math.log(vol))
18         log_impact_known.append(math.log(abs(delta_p[i])))
19
20 #Now, we need to do the regression, according to the formulas upside :
21 n = len(log_v_known)
22 mean_X = sum(log_v_known) / n
23 mean_Y = sum(log_impact_known) / n
24
25 # We initialize the numerator and denominator of Linear Regression estimator
26 :
27 num = 0.0
28 den = 0.0
29
30 for i in range(n):
31     dx = log_v_known[i] - mean_X
32     dy = log_impact_known[i] - mean_Y
33     num += dx * dy
34     den += dx * dx
35
36 # LR estimators :
37 r_estimated = num/den

```

Listing 6: Estimation of parameters S and r

Now that we estimated r and S, we need to estimate G, the impact function. We use the matricial form for the linear regression, with TX^*X and TX^*Y . We use the solver of numpy to solve it after calculating the matrixes.

```

1 # Mean volume for interpolation (same logic as before) :
2 volumes_known = np.array([d[2] for d in data if d[2] is not None])
3 mean_volume = volumes_known.mean()
4
5 # As written before, we need to compute chi,
6 # according to the parameters already estimated.
7 # The explained variable is delta (P), while the explaining variables are
8 chi1, chi2...
9
10 chi = np.zeros(len(data))
11 for i in range(len(data)):
12     epsilon = data[i][3]
13     vol = data[i][2]
14     if vol is None:
15         vol = mean_volume
16     chi[i] = epsilon * s_mean * (vol ** r_estimated)
17
18 # We set 25 as an optimal number of G computations. Indeed, G is supposed
19 # to be decreasing, it will fastly rush approximatly 0, by definition of the
20 impact function
21 k = 25
22
23 # Number of observations
24 T = len(delta_p)
25 # Response vector
26 Y = np.array(delta_p[k:])
27 X = np.zeros((T - k, k + 1))

```

```

26
27 # The first loop is for the time:
28 for i in range(T - k):
29     # The second loop are the observations k :
30     for j in range(k + 1):
31         X[i, j] = chi[k + i - j]
32
33 # And finally, we effectuate our regression estimation:  $g = (X^T X)^{-1} X^T Y$ 
34 XtX = X.T@X
35 XtY = X.T@Y
36 g_estimated_np = np.linalg.solve(XtX, XtY)

```

Listing 7: Estimation of parameter G, impact function

6.5 4. Empirical Results and Model Analysis

After executing the OLS regression and data processing on the TD4 dataset, we obtained the following empirical parameters for the Bouchaud impact model:

Estimated Impact Parameters

Parameter	Value	Description
Average Spread $\hat{S}$	0.10052	Fixed transaction cost
Volume Exponent $\hat{r}$	0.63479	Impact concavity measure
Immediate Propagator G_0	0.03226	Instantaneous impact

6.5.1 Interpretation of the Results

1. Analysis of the Spread S:

The estimated spread is: $S = 0.1005$

This value represents the price paid when placing an order, regardless of its size: all orders are subject to a fee, there is a minimum execution cost, and this value is approximately 0.1005. This value is non-negative, close to 0, but not negligible: this seems consistent.

2. Analysis of the Concavity Exponent r:

We observe a value of r equal to $r = 0.6348$.

We must verify concavity: $r < 1$, so the model observes a non-linear impact on volume. This shows us that the price does not increase proportionally to volume. On the contrary, the impact of each additional share traded decreases.

Furthermore, we observe a value not far from 0.5, which seems consistent. A concavity of 0.63 indicates moderate concavity: this means that large transactions have a significant impact on prices, compared to other markets where the observed value would be 0.5.

3. Analysis of the Propagator G, the impact function

The estimation of G gives us a great overview of the impact of past transactions on the price today. We find :

```

Estimated propagator coefficients G :
G( 0 ) = 0.03226023186950077
G( 1 ) = -0.0015578127228221177
G( 2 ) = -0.0012582104440751456
G( 3 ) = 4.558498118564518e-05
G( 4 ) = 0.00020936878057724667

```

Figure 4: 4 first price impacts G

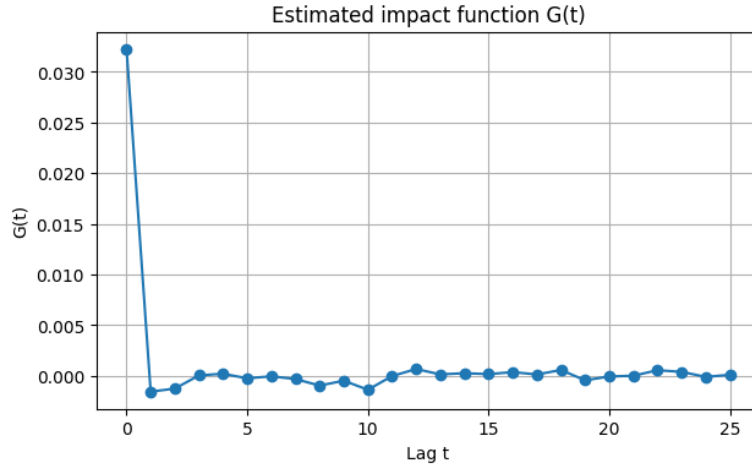


Figure 5: Estimation of G in function of t

As we can see, the first impact is higher than the other, with a value of : $G(0) = 0.032$. The following ones are close to 0. It seems that only the most recent transaction impacts a lot the price.

Furthermore, we observe negative ones, which represents a mean reverting price, which trends to match its mean after a transaction. Then, we can observe a big gap between the first impact and the following ones. G seems to be decreasing, which is a great point knowing we want to show that the model is well specified.

Therefore, we can conclude that the price impacts is only due to the last transaction, other ones are only noise. From the lag 3 to the 10th, the impact of a single trade is almost entirely transitory and disappears within a few steps.

6.5.2 Model Specification Verdict

Based on the empirical results we just computed, we can evaluate the specification of the model:

1. Well specified Concavity r: The condition $r < 1$ is satisfied. There is no linear impact on volume. The model doesn't overestimate the impact of big trades.
2. Well specified impact function: The rapid decay of $G(t)$ confirms the idea of a well specified model. The model shows a strong resilience: it means that the price impact does not become permanent, as we could see with our results. As told in the introduction, this is a key requirement for the Bouchaud framework.

Final Conclusion: The model is well-specified: concavity and G decrease are verified. The market is consistent with no arbitrage conditions. It also allows a manager to conclude

that splitting a large order X is efficient : due to the non linear scaling of volume, and also the rapid resilience of the price.

7 PART V: MULTIREOLUTION CORRELATION AND LONG-MEMORY ANALYSIS

a) Multiresolution Correlation Using Haar Wavelets

► Problem Statement: Objective

With Haar wavelets and the dataset provided with TD5, determine the multiresolution correlation between all the pairs of FX rates, using GBPEUR, SEKEUR, and CADEUR (work with the average between the highest and the lowest price and transform this average price in returns on the smallest time step). Do you observe an Epps effect and how could you explain this?

7.1 Theoretical framework

7.1.1 Wavelets

Σ Mathematical Framework: Haar Wavelet: Definition and Properties

First, what is a wavelet ? A wavelet is a function ψ belonging to the space $L^1(\mathbb{R}) \cap L^2(\mathbb{R})$, satisfying the following conditions:

$$\int_{\mathbb{R}} |\psi(t)| dt < \infty, \quad \int_{\mathbb{R}} |\psi(t)|^2 dt < \infty. \quad (16)$$

The Haar wavelet is the simplest example of a Daubechies wavelet, characterized by a single vanishing moment. It is defined using indicator functions as:

$$\psi^{Haar}(t) = \mathbf{1}_{[0, 1/2)}(t) - \mathbf{1}_{[1/2, 1)}(t). \quad (17)$$

The associated scaling function is given by:

$$\phi(t) = \mathbf{1}_{[0, 1)}(t). \quad (18)$$

The Haar wavelet allows an intuitive interpretation of multiresolution analysis: the detail coefficients measure local differences, while the scaling function captures local averages.

7.1.2 Scale-Dependent Correlation

Σ Mathematical Framework: Definition of Scale-Dependent Correlation

The wavelet correlation at scale j is obtained by normalizing the covariance, by the corresponding scale dependent variances:

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}}. \quad (19)$$

This correlation measures the movement between assets at the time horizon associated with scale j .

7.1.3 Which data

For this question, we use the dataset provided in TD5, corresponding to the low and high prices at more than 10,000 dates, spaced every 15 minutes. We have access to three stock prices: GBPEUR, SEKEUR, and CADEUR :

GBPEUR Curncy			SEKEUR Curncy			CADEUR Curncy		
Date	HIGH	LOW	Date	HIGH	LOW	Date	HIGH	LOW
07/03/2016 09:00	1,2932	1,2917	07/03/2016 09:00	0,10725	0,1072	07/03/2016 09:00	0,6842	0,6829
07/03/2016 09:15	1,294	1,293	07/03/2016 09:15	0,10728	0,10717	07/03/2016 09:15	0,6849	0,6841
07/03/2016 09:30	1,2943	1,2922	07/03/2016 09:30	0,10726	0,10719	07/03/2016 09:30	0,6844	0,6837
07/03/2016 09:45	1,293	1,2913	07/03/2016 09:45	0,10728	0,10721	07/03/2016 09:45	0,6844	0,6839
07/03/2016 10:00	1,2931	1,2921	07/03/2016 10:00	0,10725	0,10719	07/03/2016 10:00	0,684	0,6835

Figure 6: Caption

7.2 Algorithmic Implementation

7.2.1 Methodology

We need to determine the multiresolution correlation between all pairs: we will use the Haar wavelet decomposition. In order to do this, we will split the price returns into components at different time scale.

At each time scale j , we will compute the details $d_{j,k}$: this is the Haar wavelet decomposition. The details gives us information on local variations of series X and Y . Small j means small time scale so microvariations : it is full of noise. A big j means slow variations, at a bigger time scale, so there is less noise. The details represent the fluctuations of the series at a considered time scale.

Then, for each scale, we will compute ρ , the correlation between X and Y : this is our multiresolution correlations. As we compute the correlation between details, we will compute the dependance between them at a precise time scale.

We will repeat those steps for several time scale, from small ones to very big ones : as the time scale increases as well as the correlation ρ , it means we will see a Epps effect.

The Haar decomposition formulas :

At each resolution level, the Haar wavelet decomposition relies on the following coefficients.

The detail coefficients, capturing local fluctuations at scale j , are defined by:

$$d_{j,k} = \frac{x_{2k} - x_{2k-1}}{\sqrt{2}}. \quad (20)$$

The approximation coefficients, used to build the series at the next coarser scale, are given by:

$$a_{j+1,k} = \frac{x_{2k} + x_{2k-1}}{\sqrt{2}}. \quad (21)$$

These formulas are applied iteratively to construct the multiresolution representation used in the algorithmic implementation.

7.2.2 Algorithmic implementation

With the same algorithm as in the previous question, we import our dataset a way we can have :

```
1 gbp = np.array(gbp)
2 sek = np.array(sek)
3 cad = np.array(cad)
```

containing our dataset.

```
1 # First, we compute the mid prices for gbp, sek and cad, thanks to the high
  and low prices given.
2 P_gbp = 0.5 * (gbp[:,0] + gbp[:,1])
3 P_sek = 0.5 * (sek[:,0] + sek[:,1])
4 P_cad = 0.5 * (cad[:,0] + cad[:,1])
5
6 # Then, we calculate the log returns :  $R_t = \log(P_t) - \log(P_{t-1})$ 
7 R_gbp = np.diff(np.log(P_gbp))
8 R_sek = np.diff(np.log(P_sek))
9 R_cad = np.diff(np.log(P_cad))
```

```
1 gbp = np.array(gbp)
2 sek = np.array(sek)
3 cad = np.array(cad)
```

```
1 # Here, we ensure the same length : all our series need to have the same
  length.
2 m = min(len(R_gbp), len(R_sek), len(R_cad))
3 R_gbp = R_gbp[:m]
4 R_sek = R_sek[:m]
5 R_cad = R_cad[:m]
6
7 # "details" will stock the details coefficients as seen in class.
8 # Cur is the current series we will compress progressively.
9 details_gbp = []
10 details_sek = []
11 details_cad = []
12 cur_gbp = R_gbp.copy()
13 cur_sek = R_sek.copy()
14 cur_cad = R_cad.copy()
```

Now, we can finally apply by hand the Haar decomposition :

```
1 # A Haar wavelet need pairs, so we will force a paired length for each
  iteration:
2 while len(cur_gbp) >= 2:
3     n = min(len(cur_gbp), len(cur_sek), len(cur_cad))
4     n = n - (n % 2) #We make it even
5     cur_gbp = cur_gbp[:n]
6     cur_sek = cur_sek[:n]
7     cur_cad = cur_cad[:n]
8     # We stop when we arrive at 2 points or less:
9     if n < 2:
10         break
11     #And finally, we calculate the details coefficients, for Gbp, sek, and
      cad :
12
13     # Foreach detail, we apply the formula :  $d_{j,k} = (x_{2k} - x_{2k-1})/\sqrt{2}$ 
14     d_gbp = (cur_gbp[0::2] - cur_gbp[1::2]) / np.sqrt(2.0)
15     d_sek = (cur_sek[0::2] - cur_sek[1::2]) / np.sqrt(2.0)
```

```

16 d_cad = (cur_cad[0::2] - cur_cad[1::2]) / np.sqrt(2.0)
17 # And we add it in the details stock
18 details_gbp.append(d_gbp)
19 details_sek.append(d_sek)
20 details_cad.append(d_cad)
21 # Then, we move on the next scale, with the formula :  $a_{j+1,k} = (x_{2k} +$ 
     $x_{2k+1})/\sqrt{2}$ 
22 cur_gbp = (cur_gbp[0::2] + cur_gbp[1::2]) / np.sqrt(2.0)
23 cur_sek = (cur_sek[0::2] + cur_sek[1::2]) / np.sqrt(2.0)
24 cur_cad = (cur_cad[0::2] + cur_cad[1::2]) / np.sqrt(2.0)
25 # So, the series is smooth now, each iteration corresponds to a larger
    horizon as the previous one.

```

The last part consist on computing the Pearson correlation

```

1 # Finally, we are ready to compute the pearson correlation between
2 # each data with the onther ones, with the same scale at each iteration.
3 rho_gbp_sek = []
4 rho_gbp_cad = []
5 rho_sek_cad = []
6 for j in range(len(details_gbp)):
7     # We compute the correlation :  $\rho = \text{corr}(dj0, dj1)$ 
8     rho_gbp_sek.append(np.corrcoef(details_gbp[j], details_sek[j])[0, 1])
9     rho_gbp_cad.append(np.corrcoef(details_gbp[j], details_cad[j])[0, 1])
10    rho_sek_cad.append(np.corrcoef(details_sek[j], details_cad[j])[0, 1])
11 # We want the results for each horizon 15, 30, 60, 120... 15~k minutes.
12 base_minutes = 15

```

7.3 Empirical results and Model Analysis

7.3.1 Empirical results

After applying the Haar Discrete Wavelet Transform (DWT) on the FX return series (base step of 15 minutes), we computed the multiresolution correlations across 13 scales. The table below summarizes the core results for short, medium, and long-term horizons:

Wavelet Correlation Results (ρ_j)

Scale / Horizon	GBP-SEK	GBP-CAD	SEK-CAD	Observation
j=1 : 15 min	0.1600	0.2181	0.1743	High-frequency noise
j=5 : 4 hours	0.1127	0.3096	0.2835	Intermediate scale
j=10 : 128 hours	0.8223	0.0824	0.2411	Fundamental correlation
j=11 : 256 hours	-0.6348	-0.1794	0.7148	Statistical instability

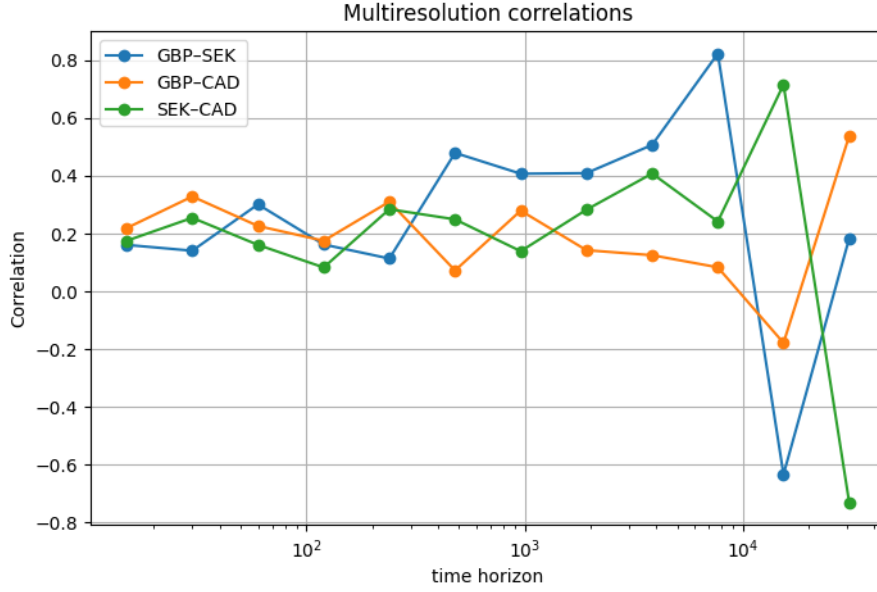


Figure 7: Multiresolution correlations

7.3.2 Interpretation of the Results

Σ Mathematical Framework: The Epps Effect

As the sampling frequency increases (meaning Delta t converges to 0), the measured correlation tends to zero:

$$\lim_{\Delta t \rightarrow 0} \rho_{X,Y}(\Delta t) = 0 \quad (22)$$

And in the other hand, as Delta t increases (longer horizons like hours or days), the correlation converges toward a real value, representing the real correlation at long time scale.

It seems that only one pair shows an Epps effect: the GBP vs SEK. Indeed, we observe a correlation from 0.16 to 0.82 from time scale 1 to 10. This is the Epps effect: as the time scale increases, the correlation follows by increasing.

We can explain it by the fact that at time scale 1, which represents 15 minutes, the correlation captures only noise. There is no relevant market information to take into account. However, at $j = 10$, the correlation captures larger volumes of details, which enables the correlation to be more precise and to avoid noise at long term. We observe an Epps effect. On the other hand, for the 2 other pairs, we can't observe an Epps effect.

We can interpret it a financial way: the Canadian dollar might be affected by big shocks, as it is linked with commodities like petrol or oil prices.

At time scale gets bigger than 10, we can observe wrong results, which doesn't represent the market at all. The correlation goes from 0.82 to -0.6, which represents a lack of data. It is biased.

Conclusion : We applied a Haar wavelet decomposition on our mid price returns. Then, we calculated, for each time scale $j = 15$ minutes, the Pearson correlation between all details with each other. So we obtained our multi resolution correlation at a specific time horizon.

On small scales ($j=15$ minutes), the correlation observed were very low : the noise is too important at this time scale. The results are consistent.

As we increase the time scale, we observed that the correlation GBP-SEK increased, from 0.16 to 0.82. The correlation captures less noise : this is the Epps effect. As time scale increases, the correlation increases.

However, for our two other pairs, the correlation calculated doesn't increase. We can't see the Epps effect. This might be explained by the fact that GBP and SEK are more correlated due to the geographical reasons, while the CAD might be influenced mainly by shocks on commodities for examples oil or petrol. It doesn't show any correlation with european currencies.

b) Hurst Exponent and Annualized volatility using daily volatility

► Problem Statement: Objective

Goal: Calculate the Hurst exponent of GBPEUR, SEKEUR, and CADEUR. Determine their annualized volatility using the daily volatility and Hurst exponents.

7.4 Theoretical framework

7.4.1 Volatility scaling and Hurst exponent

Σ Mathematical Framework: Annualizing volatility with the Hurst exponent

In practice, volatility is often annualized using the square-root-of-time rule:

$$\sigma_T = \sigma_{\Delta t} \sqrt{T},$$

which relies on the assumption that returns are Gaussian, independent and identically distributed. This corresponds to the case of a standard Brownian motion.

More generally, when returns exhibit long-range dependence, volatility does not scale with $\sqrt{T}$ but rather with a power law:

$$\sigma_T = \sigma_{\Delta t} T^H,$$

where $H \in]0, 1[$ is the Hurst exponent.

The value of H characterizes the nature of the process:

- $H = \frac{1}{2}$: the process is a Brownian motion and the square-root-of-time rule applies.
- $H > \frac{1}{2}$: the process is persistent, leading to an underestimation of long-term risk when using $\sqrt{T}$.
- $H < \frac{1}{2}$: the process is anti-persistent, leading to an overestimation of long-term risk.

7.4.2 Estimation of H: The Multi-resolution Estimator

To estimate the Hurst exponent H from high-frequency data, we compare the second-order moments of returns at two different scales: the original resolution N and a halved resolution $N/2$.

Σ Mathematical Framework: The M_2 and M'_2 Estimators (Eq. 4)

Let $X(t)$ be the price process. We define M_2 as the average of squared increments at the highest available frequency:

$$M_2 = \frac{1}{NT} \sum_{i=1}^{NT} |X(i/N) - X((i-1)/N)|^2 \quad (23)$$

We then define M'_2 , the same statistic based on the same observation window but with halved resolution: we take steps of $2/N$):

$$M'_2 = \frac{2}{NT} \sum_{i=1}^{NT/2} |X(2i/N) - X(2(i-1)/N)|^2 \quad (24)$$

According to the course, the estimator $\hat{H}$ is derived from the ratio of these two moments:

$$\hat{H} = \frac{1}{2} \log_2 \left(\frac{M'_2}{M_2} \right) \quad (25)$$

This estimator converges almost surely toward the true Hurst exponent H .

7.4.3 Which data

We use the same dataset as the previous question, low and high prices for several dates for 3 assets : GBPEUR, SEKEUR, and CADEUR.

7.5 Algorithmic implementation

We apply the theoretical model defined :

```
1 # We use the price returns computed previously :
2 all_returns = [R_gbp, R_sek, R_cad]
3 all_names = ["GBPEUR", "SEKEUR", "CADEUR"]
4
5 # We loop for each asset returns :
6 for i in range(len(all_names)):
7     R = all_returns[i] # The returns at time i for each asset
8
9     # To compute M2', we decide to take only pairs returns :
10    n = len(R) if len(R) % 2 == 0 else len(R) - 1
11    R_f = R[:n] # We get all pairs returns
12
13    # then, we compute M2 and M2' according to the formulas written
14    # previously :
15    # Sum of : X(2i/N) - X(2(i-1)/N)
16
17    M2 = np.mean(R_f**2) # For M2
```

```

17 M2prime = np.mean((R_f[0::2] + R_f[1::2])**2) # For M'2
18
19 # Finally, for the hurst exponent, we compute : H = 1/2 * log2(M'2 / M2)
20 H = 0.5 * math.log2(M2prime / M2)
21
22 # We move from 15-minute intervals to 1-day intervals, which is 96
   periods, via sqrt:
23 sigma_d = np.std(R) * np.sqrt(96)
24
25 # And finally, we use the hurst exponent computed juste before to
   annualize volatility:
26 sigma_a = sigma_d * (252 ** H)

```

7.6 Empirical results and Model Analysis

7.6.1 Empirical results

Based on the multi-resolution moment estimator (M_2 and M'_2 ratio) and the scaling laws previously discussed, we obtained the following empirical parameters for the three currency pairs. The annualized volatility is calculated using the corrected fractal scaling 252^H .

Estimated Hurst and Multi-scale Volatility

Pair	Hurst Exponent ($\hat{H}$)	Daily Vol (σ_{day})	Annual Vol (σ_{ann})
GBPEUR	0.6720	0.00611	0.2511
SEKEUR	0.6538	0.003205	0.1191
CADEUR	0.6549	0.00496	0.1855

7.6.2 Interpretation of the Results

Persistence of the increments

As we can see, the Hurst exponents observed are all above 0.5 : the increments are persistent. It means that if a return is positive or negative, it has a higher probability that the next return has the same sign : there is clusters of positive returns and negative returns. This persistence might suggest that the market is not perfectly efficient at high frequencies.

Square root vs Hurst exponent

The most striking result is the massive gap between standard risk assessment and fractal scaling. We can observe a massive gap between the volatility using the square root and the hurst exponent calculated previously. Indeed : For GBPEUR, using the classical $\sqrt{252}$ rule would lead to an annual volatility of approximately $0.6111\% \times \sqrt{252} = 9.70\%$. And by incorporating the persistence ($H = 0.6720$), the volatility scales to 25.11%.

Risk analysis

From a risk perspective : standard models which uses $\sqrt{t}$ for time aggregation would underestimate the annual : 9.7% to 25%, for the British Pound in our sample.

8 GENERAL CONCLUSION

Questions A, B, C

1. **Data Engineering:** Robust parsing is a prerequisite. Naive imports would have discarded 60% of the data due to date format issues.
2. **Non-Parametric approach:** The Kernel Density estimator provided a flexible fit but proved conservative (1.57% failure rate vs 5% target), highlighting regime changes in volatility.
3. **Tail Risk Reality:** Both the Expected Shortfall ratio ($ES/VaR \approx 1.41$) and the EVT Tail Index ($\xi \approx 0.37$) converge to prove that Natixis returns are Leptokurtic. Standard Gaussian models are inadequate for this asset.

So, we moved from VaR to Expected Shortfall and integrated Extreme Value Theory. Overall, the analysis shows the limits of gaussian assumptions for market estimation. Non parametric VaR, ES and EVT reveal heavier tails and severe losses than predicted by standard models. It shows the importance of taking into account the tail returns when assessing risk on financial data.

Questions D, E

1. Bouchaud's parameters estimation

First, we estimated Bouchauds parameters : S , bid ask spread, r , the concavity, and G , the impact function. As results, we found a non negligible concavity, and a decreasing impact function G : the model is well specified, it doesn't permit arbitrage. From a practical perspective, this model supports the idea that : splitting large orders, is an efficient execution strategy in liquid markets.

2. Multiresolution correlations

Then, we analysed the multiresolution correlation using Haar wavelet and analysed the Epps effect. The correlations between FX rates depends strongly on the choice of time horizons. With a small time horizon (for example 15 minutes), the model only captures noise : there is no relevant correlation between FX rates. However, as time scale increases, correlation is relevant : GBP and SEK seem to see there correlation increasing : this is the Epps effect. No Epps effect was observed regarding the CAD FX rates correlations. This is highlighting that correlation dynamics are asset-dependent and influenced by underlying economic factors.

3. Hurst exponents estimation

Finally, we computed the Hurst exponents : the increments are persistent ($H > 0.5$). in contrary to a classic random walk, there is clusters of positive and negative price returns in the series. Then, we annualized volatility : we observed that using the hurst exponent, annualized volatility are significantly higher than when using the classical square root.